

Dokumentasi Praktikum Pemrograman Mobile

Pertemuan 3



Nama:

Dimas Anugrah Saputra

NIM:

20240801064

UNIVERSITASA ESA UNGGUL

TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

1. Pendahuluan

Pada praktikum ini dipelajari cara menggunakan TextField, TextEditingController, serta Form dan TextFormField untuk menerima data dari pengguna sekaligus melakukan validasi terhadap input tersebut. Validasi digunakan untuk memastikan data yang dimasukkan sudah sesuai dengan aturan sebelum digunakan atau diproses oleh aplikasi.

Selain itu, praktikum ini juga membahas konsep **state**, yaitu data yang nilainya dapat berubah dan memengaruhi tampilan aplikasi. Untuk aplikasi sederhana, perubahan state dapat dikelola menggunakan setState(). Namun, jika data digunakan oleh beberapa widget atau halaman, pengelolaannya menjadi lebih rumit. Oleh karena itu, digunakan ChangeNotifier dan Provider sebagai pengenalan dasar dalam **state management**.

2. Tujuan Praktikum

- Memahami cara menggunakan TextField dan TextEditingController untuk menerima input dari pengguna.
- Membuat form yang dilengkapi dengan proses validasi data.
- Mengetahui perbedaan antara penggunaan setState() dengan konsep state management.
- Menerapkan ChangeNotifier dan Provider untuk mengelola state aplikasi.
- Memahami fungsi context.watch() dan context.read() dalam mengakses state.

3. Teori dasar

- Input dan Form: TextField digunakan untuk menerima teks dari pengguna, sedangkan data yang dimasukkan dapat diakses menggunakan TextEditingController. Jika terdapat beberapa input yang perlu diperiksa, dapat menggunakan Form dengan GlobalKey<FormState> kemudian menjalankan validate() untuk melakukan validasi.
- State: Merupakan data yang dapat mengalami perubahan dan perubahan tersebut dapat memengaruhi tampilan aplikasi.
- Provider: Digunakan untuk menempatkan objek state yang berasal dari ChangeNotifier pada bagian atas widget tree. Widget yang membutuhkan data tersebut dapat mengaksesnya, dan ketika notifyListeners() dipanggil, widget yang terkait akan diperbarui secara otomatis.

4. Langkah praktikum

Buat projek baru flutter di terminal:

```
flutter create praktikum1
```

```
cd praktikum3
flutter run
```

5. Implementasi textfield

Pada tahap pertama, dibuat halaman input sederhana yang memungkinkan pengguna memasukkan nama. Nilai input disimpan menggunakan `TextEditingController`, kemudian ditampilkan sebagai sapaan ketika tombol ditekan. Modul menggunakan `StatefulWidget` karena terdapat data `_salam` yang berubah setelah tombol ditekan.

```
class _InputPageState extends State<InputPage> {
  final _controller = TextEditingController();
  String _salam = '';

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }
}
```

`_controller` berfungsi untuk mengambil dan membaca teks yang dimasukkan pengguna melalui `TextField`. Sementara itu, `_salam` digunakan untuk menyimpan pesan sapaan yang nantinya ditampilkan pada halaman. Kemudian, `dispose()` digunakan untuk membersihkan dan melepaskan `TextEditingController` setelah halaman tidak digunakan lagi.

Selanjutnya, input nama dibuat menggunakan `TextField`:

```
TextField(
  controller: _controller,
  decoration: const InputDecoration(
    labelText: 'Nama',
    border: OutlineInputBorder(),
  ), InputDecoration
```

Kode tersebut menghubungkan `TextField` dengan `_controller` agar teks yang dimasukkan pengguna dapat diambil. `labelText` digunakan untuk memberikan keterangan pada kolom input bahwa pengguna perlu memasukkan nama, sedangkan `OutlineInputBorder` digunakan untuk memberikan tampilan garis pada bagian input.

6. Implementasi form dengan validasi

Tahap berikutnya adalah membuat form yang memiliki beberapa input, yaitu nama lengkap, email, jurusan, dan persetujuan pengguna. Modul menggunakan `Form` dengan `GlobalKey<FormState>` agar seluruh input dapat divalidasi ketika tombol Daftar ditekan.

```
class _FormPageState extends State<FormPage> {
  final _formKey = GlobalKey<FormState>();
  final _nama = TextEditingController();
  final _email = TextEditingController();

  String? _jurusan;
  bool _setuju = false;
}
```

- `_formKey` digunakan untuk mengidentifikasi form.
- `_nama` digunakan untuk mengontrol input nama.
- `_email` digunakan untuk mengontrol input email.
- `_jurusan` menyimpan pilihan jurusan.
- `_setuju` menyimpan status checkbox persetujuan.

Apabila semua proses validasi berhasil dilakukan, aplikasi akan menampilkan `SnackBar` yang berisi informasi mengenai nama dan jurusan pengguna.

```
TextFormField(
  controller: _nama,
  decoration: const InputDecoration(
    labelText: 'Nama lengkap',
    border: OutlineInputBorder(),
  ),
  validator: (v) =>
    (v == null || v.trim().isEmpty)
      ? 'Nama wajib diisi'
      : null,
),
```

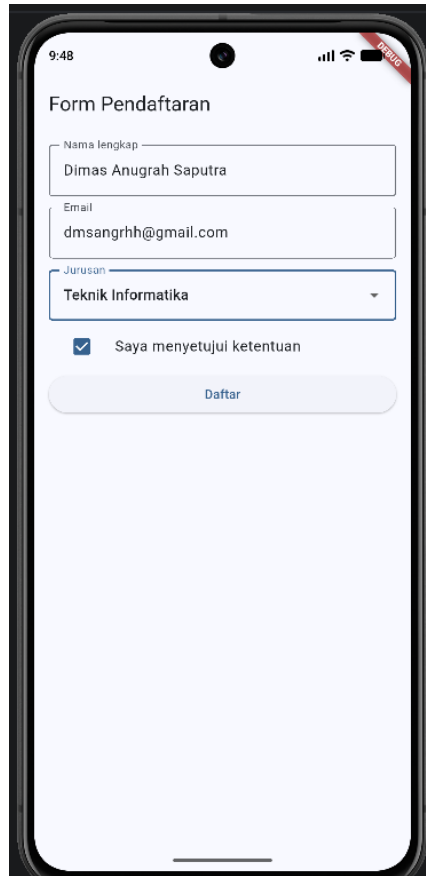
Pada bagian validator, program akan mengecek apakah kolom nama sudah diisi atau masih kosong. Jika kolom tersebut kosong, akan muncul pesan **"Nama wajib diisi"**.

Namun, jika input sudah diisi dengan benar, validator akan mengembalikan nilai null.

```
TextFormField(  
  controller: _email,  
  keyboardType: TextInputType.emailAddress,  
  decoration: const InputDecoration(  
    labelText: 'Email',  
    border: OutlineInputBorder(),  
  ),  
  validator: (v) {  
    if (v == null || !v.contains('@')) {  
      return 'Email tidak valid';  
    }  
    return null;  
  },  
)
```

Validasi email dilakukan dengan mengecek apakah data yang dimasukkan mengandung karakter @. Jika karakter tersebut tidak ditemukan, maka aplikasi akan menampilkan pesan "Email tidak valid".

Hasil akhir:



7. Mengapa butuh State Management?

Pada Bagian A dan B, state masih dikelola dalam satu halaman menggunakan `setState()`. Misalnya pada aplikasi daftar tugas, terdapat halaman yang menampilkan daftar tugas dan halaman lain untuk menambahkan tugas baru. Kedua halaman tersebut membutuhkan data yang sama, sehingga jika data dikirim melalui constructor dan callback, pengelolaannya bisa menjadi lebih rumit. Untuk mengatasinya, data dapat ditempatkan dalam satu objek bersama yang bisa digunakan oleh beberapa halaman. Dalam hal ini, Provider digunakan untuk membantu mengelola dan membagikan state tersebut.

8. Implementasi provider

Sebelum menggunakan provider, di terminal folder proyek, pasang paket dengan perintah:

```
flutter pub add provider
```

```
PS C:\Users\LENOVO LOQ\pemmob\pert3\praktikum3> flutter pub add provider
Resolving dependencies...
Downloading packages...
  cupertino_icons 1.0.9 (2.0.0 available)
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
+ nested 1.0.0
+ provider 6.1.5+1
  test_api 0.7.12 (0.7.14 available)
```

```
class TugasModel extends ChangeNotifier {
  final List<Tugas> _items = [];

  List<Tugas> get items => List.unmodifiable(_items);

  int get jumlahSelesai =>
    _items.where((t) => t.selesai).length;
}
```

Pada kode tersebut, items digunakan untuk menyimpan daftar tugas. Data tugas diambil dari items, sedangkan jumlahSelesai digunakan untuk menghitung jumlah tugas yang telah selesai. Sementara itu, ChangeNotifier berfungsi memberi tahu widget ketika terjadi perubahan pada state.

```
void tambah(String judul) {
  _items.add(Tugas(judul));
  notifyListeners();
}

void toggle(int index) {
  _items[index].selesai = !_items[index].selesai;
  notifyListeners();
}

void hapus(int index) {
  _items.removeAt(index);
  notifyListeners();
}
}
```

Ketiga method tersebut menggunakan notifyListeners() untuk memberi tahu widget yang terkait bahwa telah terjadi perubahan pada state.

- tambah() : Digunakan untuk menambahkan tugas baru.
- toggle() : Digunakan untuk mengubah status tugas menjadi selesai atau belum selesai.

- `hapus()` : Digunakan untuk menghapus tugas dari daftar.
- `notifyListeners()` : Berfungsi memberi tahu widget bahwa terdapat perubahan pada data atau state.

```
void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => TugasModel(),
      child: const MyApp(),
    ), ChangeNotifierProvider
  );
}
```

`ChangeNotifierProvider` menyediakan `TugasModel` sehingga dapat diakses oleh halaman-halaman di dalam aplikasi.

```
void _simpan() {
  final judul = _controller.text.trim();

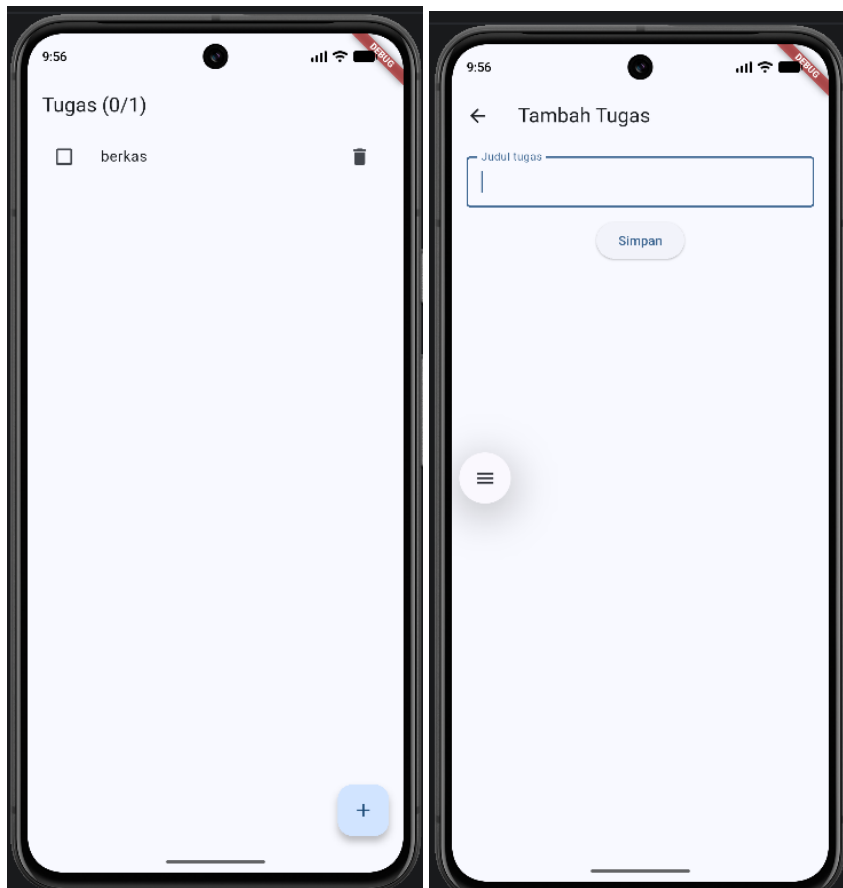
  if (judul.isEmpty) return;

  context.read<TugasModel>().tambah(judul);

  Navigator.pop(context);
}
```

- Mengambil teks dari input.
- Memastikan judul tidak kosong.
- Menambahkan data melalui `TugasModel`.
- Kembali ke halaman daftar setelah data disimpan.

Hasil akhir:



9. Latihan Mandiri

- a. Tambahkan validasi pada TambahPage: judul minimal 3 karakter dan tampilkan pesan error.

```

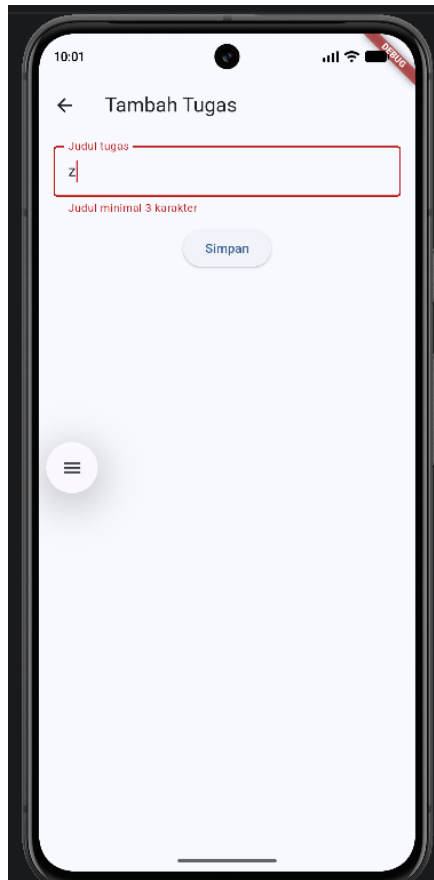
body: Form(
  key: _formKey,
  child: Padding(
    padding: const EdgeInsets.all(16),
    child: Column(
      children: [
        TextFormField(
          controller: _controller,
          autofocus: true,
          decoration: const InputDecoration(
            labelText: 'Judul tugas',
            border: OutlineInputBorder(),
          ),
          validator: (v) {
            if (v == null || v.trim().length < 3) {
              return 'Judul minimal 3 karakter';
            }

            return null;
          },
        ),
      ],
    ),
  ),
)

```

Jika kolom input kosong atau jumlah karakternya kurang dari 3, validator akan menampilkan pesan "Judul minimal 3 karakter" di bawah kolom input. Jika data sudah sesuai, validator akan mengembalikan null, yang berarti tidak terdapat kesalahan pada input.

Hasil:



- b. Tambahkan method `hapusSelesai()` pada `TugasModel` dan tombol di AppBar untuk menghapus semua tugas yang sudah selesai.

```
void hapusSelesai() {  
    _items.removeWhere((t) => t.selesai);  
    notifyListeners();  
}
```

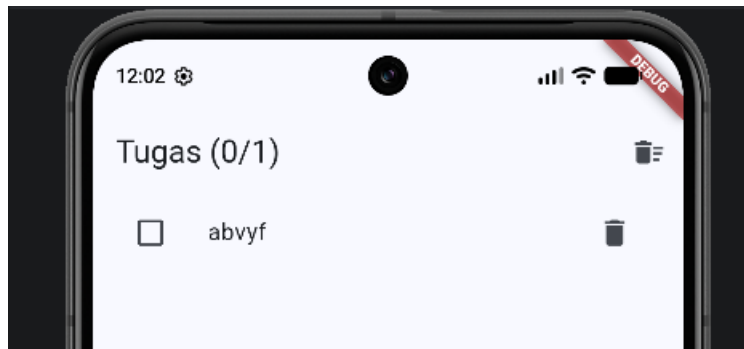
Method ini menggunakan `removeWhere` untuk menghapus seluruh tugas yang sudah berstatus selesai. Setelah proses penghapusan selesai, `notifyListeners()` dipanggil agar tampilan UI dapat diperbarui secara otomatis.

```

appBar: AppBar(
  title: Text(
    'Tugas (${model.jumlahSelesai}/${model.items.length}}',
  ), Text
  actions: [
    IconButton(
      icon: const Icon(Icons.delete_sweep),
      onPressed: () {
        context.read<TugasModel>().hapusSelesai();
      },
    ), IconButton
  ],
), AppBar

```

Bagian actions pada AppBar digunakan untuk menambahkan tombol di bagian kanan atas. Ketika tombol ditekan, onPressed akan menjalankan `context.read<TugasModel>().hapusSelesai()` untuk menghapus seluruh tugas yang sudah selesai.



Jadi, saat pengguna memilih beberapa tugas sekaligus lalu menekan ikon tempat sampah di bagian atas, seluruh tugas yang dipilih akan langsung terhapus.

- c. Tampilkan SnackBar "Tugas ditambahkan" setelah tugas disimpan.

```

void _simpan() {
  if (_formKey.currentState!.validate()) {
    final judul = _controller.text.trim();

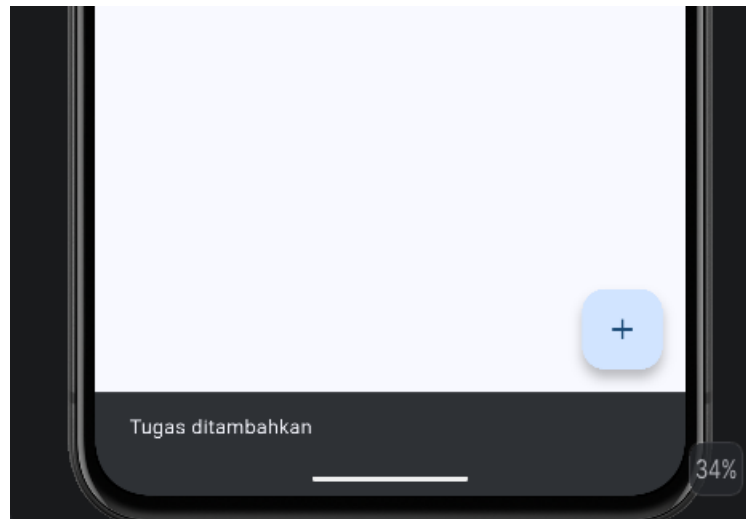
    context.read<TugasModel>().tambah(judul);

    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
        content: Text('Tugas ditambahkan'),
      ), SnackBar
    );

    Navigator.pop(context);
  }
}

```

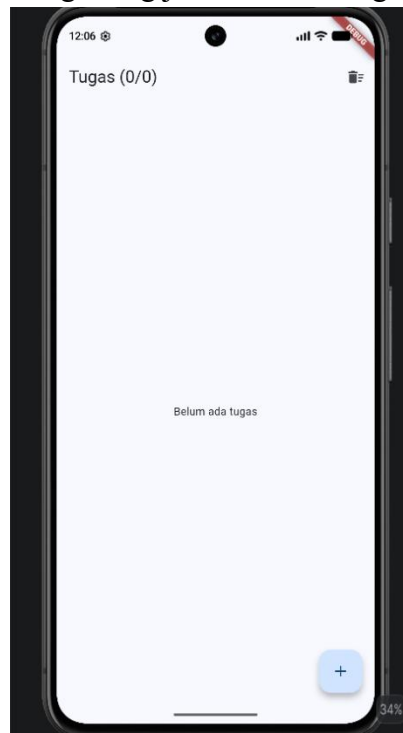
Disini, ketika sebuah tugas baru ditambahkan, sebuah snackbar otomatis muncul di layar, navigator pop membuat kembali ke halaman sebelumnya. Hasil:



- d. Bila daftar kosong, tampilkan teks "Belum ada tugas" di tengah layar.

```
body: model.items.isEmpty  
  ? const Center(  
    child: Text('Belum ada tugas'),  
  ) : Center  
  : ListView.builder(  
    // ...  
  )
```

Disini jika tidak ada item di layar utama, pesan belum ada tugas otomatis muncul di tengah layar dan akan menghilang jika membuat tugas baru.



10. Buat aplikasi Daftar Belanja dengan ketentuan:

- Halaman Form Tambah: nama barang (wajib), jumlah (wajib, angka lebih dari 0), dan kategori (dropdown), dengan validasi pada setiap isian.
- Halaman Daftar: menampilkan semua barang, dapat dicentang sebagai "sudah dibeli" dan dihapus.
- State disimpan pada satu ChangeNotifier dan dibagikan dengan Provider; AppBar menampilkan jumlah barang yang belum dibeli.

11. Implementasi Tugas

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class Barang {
  String nama;
  int jumlah;
  String kategori;
  bool sudahDibeli;

  Barang(
    this.nama,
    this.jumlah,
    this.kategori, {
    this.sudahDibeli = false,
  });
}

class BelanjaModel extends ChangeNotifier {
  final List<Barang> _items = [];

  List<Barang> get items => List.unmodifiable(_items);

  int get jumlahBelumDibeli =>
    _items.where((barang) => !barang.sudahDibeli).length;

  void tambah(String nama, int jumlah, String kategori) {
```

Bagian ini berfungsi sebagai pusat pengelolaan data dalam aplikasi. BelanjaModel menggunakan ChangeNotifier untuk menyimpan daftar barang yang dimasukkan

oleh pengguna. `items` digunakan untuk mengambil data barang, sedangkan `jumlahBelumDibeli` berfungsi menghitung jumlah barang yang belum ditandai sebagai sudah dibeli.

```
void tambah(String nama, int jumlah, String kategori) {
    _items.add(
        Barang(
            nama,
            jumlah,
            kategori,
        ),
    );
    notifyListeners();
}

void toggle(int index) {
    _items[index].sudahDibeli =
        !_items[index].sudahDibeli;

    notifyListeners();
}

void hapus(int index) {
    _items.removeAt(index);
    notifyListeners();
}
}
```

Bagian ini berisi beberapa fungsi utama untuk mengatur daftar belanja. `tambah()` digunakan untuk menambahkan barang baru, `toggle()` untuk mengubah status barang menjadi sudah atau belum dibeli, sedangkan `hapus()` berfungsi menghapus barang dari daftar. Setelah terjadi perubahan, `notifyListeners()` digunakan agar tampilan aplikasi dapat diperbarui secara otomatis.

```
void main() {  
  runApp(  
    ChangeNotifierProvider(  
      create: (_) => BelanjaModel(),  
      child: const MyApp(),  
    ),  
  );  
}
```

Bagian ini digunakan untuk menyediakan BelanjaModel agar bisa diakses oleh berbagai halaman dan widget dalam aplikasi. Dengan menggunakan Provider, pengelolaan state dapat dilakukan secara terpusat tanpa perlu membuat pengelolaan data secara terpisah di setiap halaman.

```

class _TambahBarangPageState
    extends State<TambahBarangPage> {
    final _formKey = GlobalKey<FormState>();

    final _nama = TextEditingController();
    final _jumlah = TextEditingController();

    String? _kategori;

    @override
    void dispose() {
        _nama.dispose();
        _jumlah.dispose();
        super.dispose();
    }

    void _simpan() {
        if (_formKey.currentState!.validate()) {
            context.read<BelanjaModel>().tambah(
                _nama.text.trim(),
                int.parse(_jumlah.text),
                _kategori!,
            );

            Navigator.pop(context);
        }
    }
}

```

Bagian ini digunakan untuk mengolah data yang dimasukkan melalui form tambah barang. `GlobalKey<FormState>` berfungsi untuk menjalankan validasi pada setiap input. Jika semua data sudah valid, barang akan ditambahkan ke `BelanjaModel` melalui fungsi `tambah()`, kemudian halaman form ditutup menggunakan `Navigator.pop()`. Validasi jumlah memastikan input tidak kosong, berupa angka, dan memiliki nilai lebih dari 0.

```

@override
Widget build(BuildContext context) {
  final model = context.watch<BelanjaModel>();

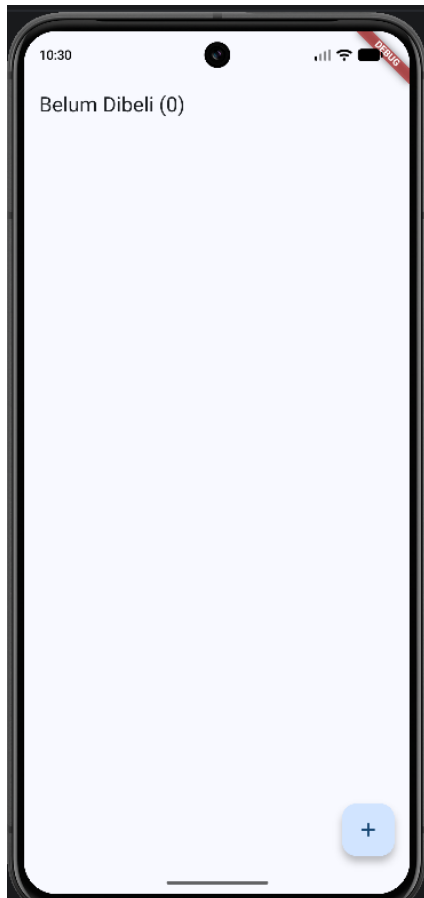
  return Scaffold(
    appBar: AppBar(
      title: Text(
        'Belum Dibeli (${model.jumlahBelumDibeli}}',
      ),
    ),
    body: ListView.builder(
      itemCount: model.items.length,
      itemBuilder: (context, index) {
        final barang = model.items[index];

        return ListTile(
          leading: Checkbox(
            value: barang.sudahDibeli,
            onChanged: (_) {
              context.read<BelanjaModel>().toggle(index);
            },
          ),
          title: Text(
            barang.nama,
            style: TextStyle(
              decoration: barang.sudahDibeli
                ? TextDecoration.lineThrough
                : null,
            ),
          ),
        );
      },
    ),
  );
}

```

Bagian ini digunakan untuk menampilkan semua barang yang terdapat di dalam BelanjaModel. `context.watch()` berfungsi memantau perubahan data sehingga tampilan dapat diperbarui secara otomatis. Checkbox digunakan untuk menandai barang yang sudah dibeli, sedangkan `context.read()` digunakan untuk menjalankan fungsi `toggle()` saat status checkbox berubah.

12. Hasil Tugas





13. Pertanyaan Refleksi

- Mengapa TextEditingController harus di-dispose?
: TextEditingController perlu di-dispose() karena menyimpan resource dan listener untuk memantau perubahan teks serta posisi kursor. Jika tidak dihapus, listener tersebut tetap aktif meskipun halaman sudah ditutup.
- Kapan cukup menggunakan setState dan kapan menggunakan Provider?
: setState() cocok digunakan untuk mengelola state yang hanya diperlukan dalam satu widget atau halaman. Sedangkan Provider lebih sesuai untuk mengelola state yang perlu digunakan dan dibagikan oleh beberapa widget atau halaman.

- Apa yang terjadi jika `notifyListeners()` tidak dipanggil?
: Tampilan UI tidak akan mengalami perubahan. Meskipun data pada model sudah berubah, seperti saat data ditambahkan atau dihapus, Provider tidak akan memberi tahu widget yang menggunakannya jika `notifyListeners()` tidak dipanggil. Akibatnya, UI tetap menampilkan data sebelumnya.
- Mengapa `onPressed` menggunakan `context.read()` bukan `context.watch()`?
: `onPressed` merupakan callback yang digunakan untuk menjalankan suatu aksi, bukan bagian dari proses build. Pada bagian ini, kita hanya perlu memanggil method pada model, seperti `tambah()` atau `hapus()`, tanpa perlu memantau perubahan state. `context.read()` digunakan untuk mengambil model tanpa membuat widget melakukan rebuild ketika terjadi perubahan data. Sedangkan `context.watch()` digunakan di dalam build ketika widget perlu memantau perubahan state dan memperbarui tampilan secara otomatis.

14. Kesimpulan

Praktikum ini berhasil menerapkan penggunaan `TextField` untuk menerima input pengguna, pembuatan form dengan validasi, serta pengelolaan state menggunakan `ChangeNotifier` dan `Provider`. Melalui aplikasi Daftar Belanja, data dapat digunakan oleh beberapa halaman dengan penyimpanan state yang terpusat dalam satu model.